

AI-Enabled Choreography: Multimodal Generation and Analysis of Dance using Semi-Supervised Learning

Daksh Mor
daksh3982@gmail.com

April 7, 2025

Abstract

The intersection of artificial intelligence and artistic creation presents unique opportunities and challenges. This paper details a system developed for AI-enabled choreography, focusing on bridging the gap between dance motion capture data and natural language descriptions. We propose a multimodal approach combining 3D motion visualization, representation learning via an autoencoder, semi-supervised labeling to overcome data scarcity, and bidirectional generation capabilities. Specifically, we employ a motion autoencoder to learn a compact 64-dimensional latent space for dance sequences. A semi-supervised strategy, leveraging manual labeling of a small subset (12.5%) combined with data augmentation and K-Nearest Neighbors (KNN) classification, is used to label the remaining motion data. A TextMapper network aligns BERT text embeddings with the motion latent space, facilitating text-to-dance generation. Conversely, dance-to-text description is achieved using KNN search within the shared embedding space. We demonstrate the system’s effectiveness through qualitative visualization of generated motions and quantitative evaluation of embedding space structure via t-SNE and labeling accuracy. The results highlight the potential of combining deep learning and semi-supervised techniques for creative applications in domains with limited labeled data.

1 Introduction

The integration of Artificial Intelligence (AI) into creative fields, particularly the performing arts like dance, offers novel tools for exploration, generation, and analysis [?, ?]. However, translating the nuances of human movement into computational representations and bridging modalities like motion and language remains a significant challenge. This work addresses the task of creating an AI-enabled choreography system capable of understanding and generating dance sequences based on textual descriptions, and vice-versa.

The primary contributions of this paper are:

- A 3D visualization pipeline for motion capture dance data using PyVista, enhanced for clarity and performance.
- A motion autoencoder architecture for learning compact latent representations of dance sequences.
- A practical semi-supervised labeling strategy employing data augmentation and KNN to handle the common scenario of limited labeled dance data.
- A multimodal framework enabling bidirectional generation: synthesizing dance motion from text descriptions and generating textual descriptions from dance sequences.

Our system follows a modular architecture, progressively building capabilities from data preprocessing and visualization to representation learning, labeling, and finally, multimodal generation, as illustrated conceptually in Figure 1. We leverage insights from recent work on AI and art [?, ?], particularly emphasizing AI as a collaborative tool for artists rather than a replacement.

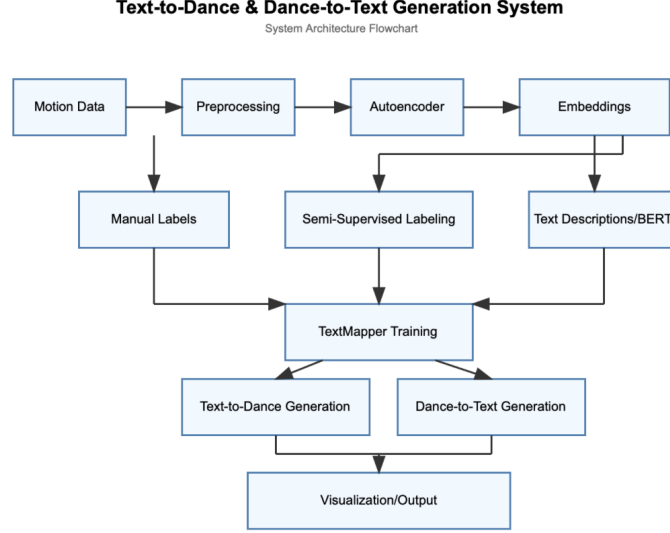


Figure 1: Conceptual flowchart of the AI-Enabled Choreography system architecture, showing data flow from preprocessing through representation learning, labeling, and multimodal generation.

2 Methods

This section details the data, preprocessing steps, and the core components of our proposed system.

2.1 Data and Preprocessing

The primary data consists of motion capture sequences stored in ‘.npy’ format. Each file represents a dance session with dimensions corresponding to joints, timesteps, and 3D spatial coordinates ($[num_joints, num_timesteps, 3]$). There are 10,000 timesteps.

The following preprocessing pipeline was applied:

1. **Time Segment Selection:** Frames 4000-8000 were extracted to focus on dynamic motion segments.
2. **Joint Selection:** The number of joints was reduced from 55 to 53 based on redundancy analysis (excluding indices 26, 53).
3. **Coordinate Adjustment:** The Z-axis was inverted for standard visualization alignment.
4. **Sequence Standardization:** Data was reshaped into 80 non-overlapping sequences, each of 50 timesteps. Joint coordinates were flattened, resulting in a feature vector of size $53 \times 3 = 159$ per timestep.
5. **Data Type Conversion:** NumPy arrays were converted to PyTorch tensors (‘torch.float’).

The final preprocessed dataset has a tensor shape of $[80, 50, 159]$.

2.2 Motion Visualization

Effective visualization is crucial for interpreting motion data. We developed a 3D skeletal animation system using the PyVista library, chosen for its performance with large datasets and 3D capabilities.

Key aspects include:

- **Skeleton Definition:** Manually defined connections between the 53 selected joints to represent a coherent human figure (Figure 2).

- **Orientation Cues:** Implemented color differentiation (red for front-facing parts) to improve directional interpretation (Figure 3).
- **Performance Optimization:** Vectorized operations and calculation reduction achieved an approx. 83% reduction in rendering time compared to initial implementations.

Animations are generated in GIF and MP4 formats (Example: Figure 4).

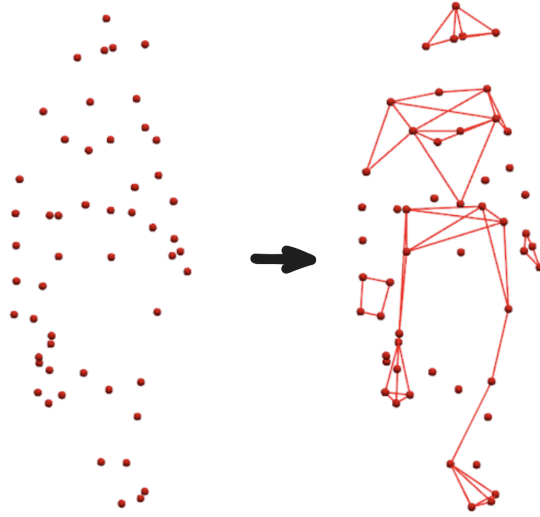


Figure 2: Diagram illustrating the defined connections between the 53 selected joints used for visualization.

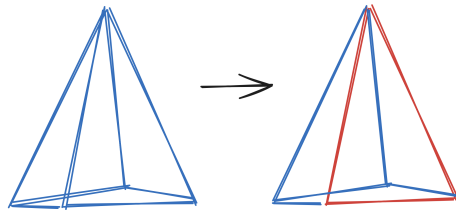


Figure 3: Illustration of the orientation indicator enhancement using color (e.g., red) to highlight front-facing skeleton parts.



Figure 4: Still frame from the generated 3D skeleton animation output.

2.3 Motion Autoencoder

To learn a compressed representation of dance dynamics, we implemented a motion autoencoder.

2.3.1 Architecture

The autoencoder consists of an encoder and a decoder, both composed of fully connected linear layers with ReLU activation functions, except for the final decoder layer which uses Sigmoid to match the normalized input range $[0, 1]$. The architecture progressively reduces dimensionality to a 64-dimensional latent space bottleneck, as shown in Figure 5.

- **Input:** Sequence tensor of shape $[50, 159]$, flattened to $[7950]$.
- **Encoder:** $\text{Linear}(7950, 1024) \rightarrow \text{ReLU} \rightarrow \text{Linear}(1024, 256) \rightarrow \text{ReLU} \rightarrow \text{Linear}(256, 64) \rightarrow \text{ReLU}$.
- **Latent Vector:** $\mathbf{z} \in \mathbb{R}^{64}$.
- **Decoder:** $\text{Linear}(64, 256) \rightarrow \text{ReLU} \rightarrow \text{Linear}(256, 1024) \rightarrow \text{ReLU} \rightarrow \text{Linear}(1024, 7950) \rightarrow \text{Sigmoid}$.

- **Output:** Reconstructed flattened sequence '[7950]', reshaped to '[50, 159]'.



Figure 5: Architecture of the Motion Autoencoder showing encoder, 64-dimensional latent space, and decoder layers.

2.3.2 Training

The autoencoder was trained using the Adam optimizer [?] with a learning rate of 0.001 and a batch size of 32. The loss function was Mean Squared Error (MSE) between the original and reconstructed sequences. Training ran for 1000 epochs with early stopping based on validation loss. (Note: Loss curve plot is omitted as per user indication).

2.4 Semi-Supervised Labeling

Addressing the absence of ground-truth labels, we adopted a semi-supervised approach:

1. **Manual Labeling:** 10 representative sequences (12.5% of the dataset) were manually assigned descriptive text labels (e.g., "neck rotation", "spin with right hand above").
2. **Feature Extraction:** The pre-trained autoencoder's encoder was used to extract 64-dimensional latent embeddings ($\mathbf{z}$) for all 80 sequences.
3. **Data Augmentation:** To improve classifier robustness, the 10 labeled sequences' embeddings were augmented using:
 - Time Warping: Simulating speed variations (factors 0.9-1.1, applied conceptually before embedding or on sequences if applicable).
 - Noise Addition: Adding Gaussian noise ($\sigma = 0.02$) to embeddings.

This generated 9 additional augmented samples per original labeled sequence. Figure 6 illustrates this.

4. **KNN Classification:** A K-Nearest Neighbors classifier ($k=2$, distance-weighted) was trained on the original 10 + 90 augmented labeled embeddings. This classifier then predicted labels for the remaining 70 unlabeled sequence embeddings. Prediction confidence was estimated using KNN's probability outputs.

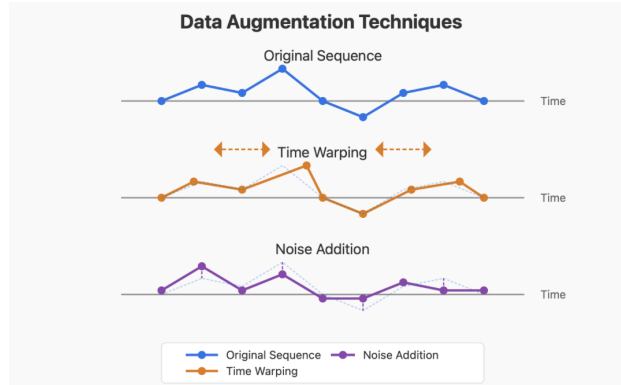


Figure 6: Conceptual illustration of data augmentation techniques (Time Warping, Noise Addition) applied to labeled sequences/embeddings.

2.5 Multimodal Mapping and Generation

To enable bidirectional translation between text and motion, we developed mapping and generation components.

2.5.1 TextMapper Model

A simple feed-forward network, termed ‘TextMapper’, was designed to map text embeddings to the motion autoencoder’s latent space.

- **Input:** 768-dimensional sentence embeddings obtained from a pre-trained BERT model (‘bert-base-uncased’) [?].
- **Architecture:** $\text{Linear}(768, 256) \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0.3) \rightarrow \text{Linear}(256, 64)$.
- **Output:** A 64-dimensional vector aligned with the motion latent space.

The TextMapper was trained using MSE loss between its output and the corresponding ground-truth motion embedding (from the labeled set), with the Adam optimizer (lr=0.001).

2.5.2 Text-to-Dance Generation

Given an input text description:

1. Encode the text using BERT.
2. Map the BERT embedding through the trained TextMapper to get a 64-dim vector $\mathbf{z}_{text}$.
3. Feed $\mathbf{z}_{text}$ into the pre-trained motion autoencoder’s decoder.
4. Visualize the resulting reconstructed motion sequence.

2.5.3 Dance-to-Text Generation

Given an input dance sequence:

1. Encode the sequence using the pre-trained motion autoencoder’s encoder to get its latent vector $\mathbf{z}_{motion}$.
2. Perform a KNN search (k=1 or k=2) in the space of known text embeddings (those corresponding to the labeled sequences) to find the text label whose embedding is closest to $\mathbf{z}_{motion}$.
3. Return the retrieved text description.

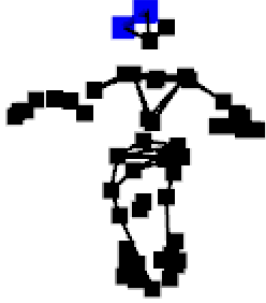
KNN was chosen for this direction due to the small size of the labeled text vocabulary (initially 10 unique labels), mitigating overfitting risks associated with training a complex sequence-to-text decoder.

3 Results

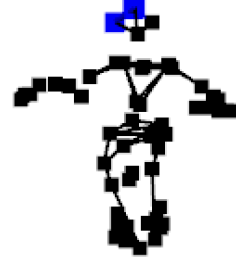
We evaluated the system components both quantitatively and qualitatively.

3.1 Autoencoder Performance

The autoencoder training converged well. Qualitative assessment of reconstructions shows that the model effectively captures the essence of the dance movements while operating in the compressed 64-dimensional latent space (Figure 7). (Note: Loss curve plot omitted).



(a) Original Dance Sequence Frame



(b) Reconstructed Sequence Frame

Figure 7: Qualitative comparison of an original dance sequence frame and its reconstruction by the autoencoder.

3.2 Semi-Supervised Labeling and Embedding Space

The quality of the semi-supervised labeling was assessed using t-SNE [?] to visualize the 64-dimensional latent embeddings of all 80 sequences, colored by their predicted labels (Figure 8). The visualization reveals discernible clusters corresponding to similar movements, suggesting the KNN approach successfully propagated labels based on latent space proximity. While most KNN predictions had high confidence (>0.7), some boundary cases exhibited lower confidence.

Table 1 compares the retrieval accuracy using only the 10 manually labeled points versus the full set with semi-supervised labels. While perfect retrieval is achieved on the small, distinct manual set, the accuracy drops when evaluated on the full dataset, reflecting the increased complexity and potential label noise introduced by the semi-supervised process on this small dataset.

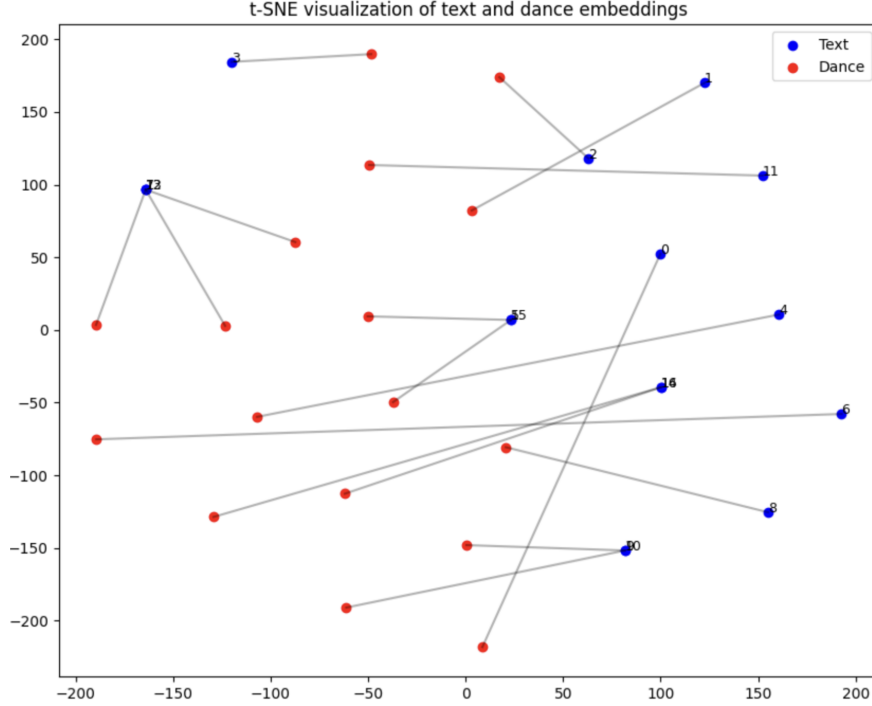


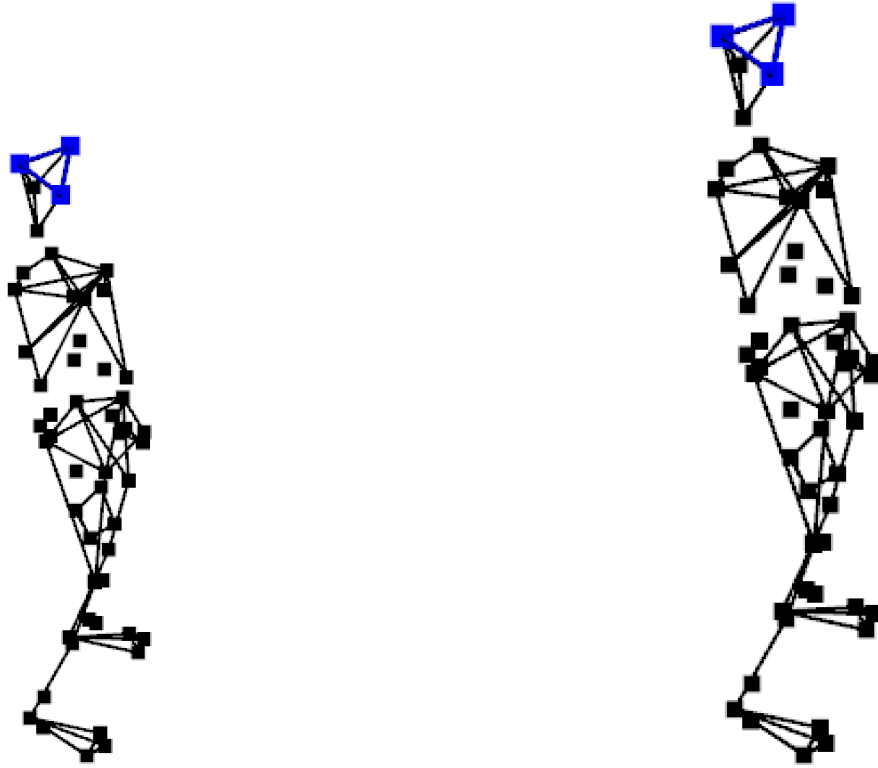
Figure 8: t-SNE visualization of the 64-dimensional latent embeddings for all 80 sequences, colored by labels obtained through semi-supervised KNN classification.

Table 1: Retrieval Accuracy Comparison on Labeled Subsets.

Labeling Method	Text→Dance Acc.	Dance→Text Acc.	Avg Acc.
Manual Only (10 seq)	1.0000	1.0000	1.0000
Semi-Supervised (80 seq)	0.5000	0.5000	0.5000

3.3 Multimodal Generation

Text-to-dance generation produced recognizable movements corresponding to input text prompts, such as "Neck Rotation" (Figure 9). The generated motion captures the core action described. Dance-to-text successfully retrieved plausible descriptions for input motion sequences using the KNN approach. The quality and specificity are inherently limited by the small vocabulary derived from the initial manual labeling phase.



(a) Generated from "Neck Rotation"

(b) Original Neck Rotation Sequence

Figure 9: Example of text-to-dance generation for the prompt "Neck Rotation" compared to an original sequence with that label.

4 Discussion

The results demonstrate the feasibility of constructing an AI-enabled choreography system using a combination of representation learning and semi-supervised techniques, even with limited labeled data.

The motion autoencoder successfully learned a compact latent space where similar movements cluster together, as confirmed by t-SNE visualization and the effectiveness of KNN classification within this space. This structured latent representation is crucial for both reconstruction and downstream tasks like labeling and generation.

The semi-supervised labeling strategy proved effective in propagating initial manual labels to the rest of the dataset. While the resulting labels are not perfect, as indicated by the t-SNE clusters and retrieval accuracy, this approach provides a practical way to bootstrap a labeled dataset in specialized domains lacking large-scale annotations. Data augmentation played a key role in improving the robustness of the KNN classifier.

The bidirectional generation capabilities highlight the potential of mapping between modalities. The TextMapper effectively bridges the gap between language and motion embeddings for text-to-dance generation. The choice of KNN for dance-to-text was pragmatic given the dataset size, leveraging the structured latent space for retrieval rather than training a potentially data-hungry generative decoder.

This work aligns with the perspective of AI as a tool to augment artistic creativity [?]. The learned latent space could potentially serve as an intuitive interface for artists to explore and manipulate movement sequences.

Limitations of this study include the small dataset size (80 sequences), the subjective nature of the

initial manual labels, and the difficulty in quantitatively evaluating the artistic quality of generated motion. The drop in retrieval accuracy after semi-supervised labeling suggests that while the approach works, label quality is critical and could be improved with more sophisticated semi-supervised methods or active learning if resources permit.

5 Conclusion and Future Work

We presented a system for AI-enabled choreography capable of visualizing dance motion capture data, learning compressed representations, performing semi-supervised labeling, and enabling bidirectional generation between text and dance. The system successfully integrates an autoencoder, KNN classification with data augmentation, and a text-embedding mapper to function effectively despite limited labeled data.

Future work could explore several directions:

- **Dataset Expansion:** Utilizing larger, more diverse motion capture datasets with professional annotations.
- **Advanced Models:** Exploring transformer-based architectures [?] for sequence modeling, or VAEs/diffusion models for more diverse generation, contingent on data availability.
- **Enhanced Artistic Control:** Incorporating finer-grained control mechanisms, potentially using structured labels like Laban Movement Analysis components.
- **Improved Evaluation:** Developing more robust metrics for evaluating the quality and expressiveness of generated dance.
- **Multi-Dancer Systems:** Extending the framework to handle interactions between multiple dancers, potentially using Graph Neural Networks.

This research provides a foundation for developing more sophisticated AI tools that can serve as creative partners in the choreographic process.